

PANDUAN SPRINT 4 - TIM BACKEND RUANG DOSEN

Periode: 24 Mei - 28 Mei 2026

Tema: Lab CRUD, Lab Submission & Grading, Testing API

Nama	Tugas	Branch
Ariel (Leader)	✔ Sudah selesai (Admin Endpoints)	-
Tio	Lab CRUD (4 endpoint)	fitur/backend-tio-lab
Sinta	Testing API via Postman & Bug Fixing	fitur/backend-sinta-testing
Dimas	Lab Submission & Grading (3 endpoint + Simulasi)	fitur/backend-dimas-lab-submission

⚠ **URUTAN Pengerjaan Wajib:** Dimas **TIDAK BOLEH** mulai coding sebelum PR (Pull Request) milik Tio sudah di-merge ke `develop` oleh Leader. Karena modul Lab Submission milik Dimas bergantung pada modul Lab CRUD milik Tio. Tio harus selesai duluan.

🔧 Aturan Git & Penamaan Branch

Format Nama Branch

Format: `fitur/backend-[nama]-[modul]`
Contoh: `fitur/backend-tio-lab` , `fitur/backend-dimas-lab-submission`

Aturan Push

JANGAN langsung push ke branch `main` atau `develop` . Yang merge ke develop cuma Leader (Ariel).

- Sebelum coding, **WAJIB** jalankan perintah sinkronisasi yang ada di bagian Persiapan Awal masing-masing tugas.
- Buat branch baru sesuai format yang ditentukan.
- Setelah selesai, push ke branch kalian sendiri, lalu buat **Pull Request ke branch** `develop` (bukan main).

Versi Node.js

Pastikan versi Node.js kalian adalah **v25.8.0**. Cek dengan perintah `node -v` di terminal. Jika berbeda, download dan install dari nodejs.org. Ini penting agar file `package-lock.json` tidak bentrok antar anggota.

Format Commit

Prefix	Digunakan untuk
<code>feat:</code>	Fitur baru
<code>fix:</code>	Perbaikan bug
<code>chore:</code>	Konfigurasi / setup

Aturan Clean Code

- Pakai nama variabel dan fungsi yang jelas. Jangan pakai nama asal kayak `x`, `data1`, `temp`.
- Satu fungsi satu tugas. Jangan campur aduk jadi satu baris panjang.
- Kasih komentar di bagian yang rumit.
- Hapus kode yang gak kepake, jangan biarin numpuk.
- Rapiin indentasi. Pakai Prettier atau ESLint yang udah ada di project.
- **Coding cuma di folder `apps/api/src/` di modul kamu. Jangan sentuh folder/file orang lain.**

👑 ARIEL (Leader) - Sprint 4 Sudah Selesai

Tidak ada branch baru

Status

✅ **SELESAI.** Seluruh tugas Sprint 4 Ariel sudah dikerjakan dan di-merge ke `develop` .

Apa yang Sudah Diselesaikan di Sprint 4

- **Endpoint** `GET /admin/statistics` - Statistik keseluruhan sistem: total user (mahasiswa, dosen, admin), total course, total kuis, total tugas.
- **Endpoint** `GET /admin/users` - Daftar seluruh pengguna dengan filter berdasarkan role.
- **Endpoint** `PUT /admin/users/:id/role` - Mengubah role pengguna (misal: menaikkan mahasiswa jadi dosen).
- **Endpoint** `DELETE /admin/users/:id` - Menghapus akun pengguna dari sistem.
- **RolesGuard & RolesDecorator** - Proteksi berlapis agar hanya Admin yang bisa mengakses endpoint di atas.

Ariel akan fokus sebagai reviewer Pull Request dari anggota tim lain pada Sprint ini.

TUGAS TIO - Lab CRUD (Praktikum)

Branch: fitur/backend-tio-lab

Deskripsi

Praktikum (Lab) adalah sesi praktik mandiri yang Dosen berikan kepada Mahasiswa di dalam mata kuliah. Dosen menulis instruksi langkah-langkah praktikum, lalu Mahasiswa membaca dan mengerjakannya secara mandiri. Tugas kamu di Sprint ini adalah membangun seluruh API manajemen praktikum dari nol: mulai dari pembuatan sesi baru, daftar praktikum, detail instruksi, sampai pengeditan dan penghapusan. Kamu juga bertanggung jawab menyiapkan tabel `LabSubmission` di database agar Dimas bisa langsung menggunakannya untuk fitur pengumpulan hasil praktikum. API yang kamu buat nanti akan dipakai oleh Frontend untuk merender halaman daftar praktikum, halaman detail instruksi, dan form kelola praktikum oleh dosen.

Langkah-Langkah

A. Persiapan Awal (Sinkronisasi Laptop)

Karena sudah banyak perubahan di `develop` (Sprint 1-3 sudah di-merge semua), laptop kamu kemungkinan besar sudah tidak sinkron. Untuk menghindari error, conflict, atau file duplikat, **WAJIB** jalankan perintah di bawah ini secara berurutan:

```
# 1. Pindah ke branch develop
git checkout develop

# 2. Ambil informasi terbaru dari GitHub tanpa menggabungkannya dulu
git fetch origin

# 3. HAPUS semua modifikasi lokal dan paksa laptop menyamakan 100% dengan GitHub
git reset --hard origin/develop

# 4. HAPUS semua file/folder asing atau duplikat yang tidak terdaftar di GitHub
git clean -fd

# 5. Sinkronkan ulang semua library agar tidak error
npm install

# 6. Masuk ke folder API dan sinkronkan database
cd apps/api
npx prisma generate
npx prisma migrate dev

# 7. Buat branch baru khusus untuk tugas kamu
git checkout -b fitur/backend-tio-lab
```

B. Pembaruan Database (Prisma)

1. Buka file skema database di `apps/api/prisma/schema.prisma` .

2. Model `PracticalLab` sudah ada di sana, kamu **tidak perlu membuatnya lagi**. Tapi kamu perlu menambahkan model baru bernama `LabSubmission`. Model ini fungsinya sama seperti `AssignmentSubmission` yang sudah ada, yaitu sebagai tempat menyimpan data pengumpulan hasil praktikum mahasiswa. Kamu bisa melihat struktur `AssignmentSubmission` di file yang sama sebagai referensi polanya.
3. Di dalam model `LabSubmission`, pastikan ada: field ID (UUID), field `labId` yang merujuk ke `PracticalLab`, field `studentId` yang merujuk ke `User`, field `fileUrl` dan `note` untuk menyimpan hasil kerja mahasiswa, field `status` bertipe String (default `"PENDING"`, nanti bisa berubah jadi `"GRADED"`), field `score` (angka) dan `feedback` (teks) untuk penilaian dosen, serta timestamp `createdAt` dan `updatedAt`.
4. Tambahkan aturan unik (unique constraint) kombinasi antara `labId` dan `studentId` agar satu mahasiswa tidak bisa submit dua kali di lab yang sama.
5. Pada kedua relasi (`lab` dan `student`), pasang argumen `onDelete: Cascade` agar saat Lab atau User dihapus, data submission-nya otomatis ikut terhapus bersih.
6. Jangan lupa tambahkan relasi balikan (array) di model `PracticalLab` dan model `User`, supaya Prisma bisa mengenali hubungan dua arah.
7. Jalankan migrasi database:

```
npx prisma migrate dev --name add_lab_submission_model
npx prisma generate
```

C. Mulai Coding

Kamu perlu membuat modul baru di dalam folder `apps/api/src/lab/`. Gunakan generator NestJS untuk membuatnya secara otomatis (jalankan dari folder `apps/api`): generate module, controller, dan service untuk `lab`. Pastikan `PrismaService` terdaftar di providers modul tersebut.

1. **Buka file service (`lab.service.ts`)**. Import `PrismaService` dan masukkan ke constructor. Kamu harus membuat 5 fungsi:
 - **Fungsi `create`** - Menerima 3 data: judul lab (`title`), instruksi panjang langkah-langkah praktikum (`instructions`), dan ID course-nya (`courseId`). Sebelum menyimpan, cek dulu apakah course-nya ada di database. Kalau tidak ada, kasih error `NotFoundException`. Lalu cek apakah user yang melakukan request adalah dosen pemilik course tersebut (bandingkan `course.instructorId` dengan `userId` dari token JWT). Kalau bukan pemilik, tolak dengan `ForbiddenException`. Kalau semua lolos, simpan data lab baru ke tabel `PracticalLab`.
 - **Fungsi `findAll`** - Ambil semua data lab dari database. Sediakan opsi filter berdasarkan parameter URL `courseId` (contoh: `?courseId=xxx`), agar frontend bisa menampilkan daftar praktikum untuk mata kuliah tertentu saja. Return array berupa id, judul, dan tanggal pembuatan lab.
 - **Fungsi `findOne`** - Ambil detail satu lab berdasarkan ID. Tujuan endpoint ini adalah agar mahasiswa bisa membaca panduan teknis praktikum. Return judul dan teks instruksi tata cara praktikum secara utuh. Kalau ID tidak ditemukan, kasih error `NotFoundException`.
 - **Fungsi `update`** - Menerima perubahan judul dan/atau instruksi lab. Sebelum mengeksekusi update, wajib validasi bahwa user yang melakukan request adalah dosen pemilik course (sama seperti fungsi create). Kalau bukan, tolak dengan `ForbiddenException`.
 - **Fungsi `remove`** - Hapus lab berdasarkan ID. Validasi kepemilikan dosen terlebih dahulu. Karena kamu sudah memasang `onDelete: Cascade` di model `LabSubmission`, saat lab dihapus maka

seluruh riwayat pengumpulan tugas mahasiswa terkait lab tersebut akan ikut lenyap otomatis, kamu tidak perlu menghapusnya satu per satu.

2. **Buka file controller (`lab.controller.ts`).** Masukkan `LabService` ke constructor. Buat 5 route endpoint:

- `POST /labs` - Buat praktikum baru. Pasang guard JWT agar hanya user yang sudah login yang bisa mengakses.
- `GET /labs` - Daftar praktikum. Terima query parameter `courseId` yang bersifat opsional.
- `GET /labs/:id` - Detail instruksi praktikum.
- `PUT /labs/:id` - Edit praktikum. Pasang guard JWT.
- `DELETE /labs/:id` - Hapus praktikum. Pasang guard JWT.

3. **Pastikan modul terdaftar.** Buka `apps/api/src/app.module.ts` . Jika kamu menggunakan generator NestJS, modul biasanya sudah otomatis terdaftar. Kalau belum, tambahkan `LabModule` ke dalam array `imports` .

D. Selesai? Push ke GitHub

```
git add .
git commit -m "feat: lab crud dan model lab submission untuk praktikum"
git push origin fitur/backend-tio-lab
```

Buka GitHub, buat **Pull Request ke branch `develop`** (bukan main). Tugaskan Ariel sebagai reviewer.

TUGAS SINTA - Testing API via Postman & Bug Fixing

Branch: fitur/backend-sinta-testing

Deskripsi

Di Sprint sebelumnya, seluruh endpoint Course dan Material sudah dibangun (termasuk endpoint `GET /courses/my` untuk menampilkan daftar matkul yang diikuti mahasiswa). Tugas kamu di Sprint ini adalah melakukan **pengujian menyeluruh** terhadap semua endpoint buatanmu tersebut menggunakan aplikasi Postman. Kamu bertanggung jawab memastikan bahwa setiap API benar-benar berfungsi sesuai spesifikasi, response JSON-nya rapi dan konsisten, serta pesan error HTTP (seperti 400, 403, 404) dikeluarkan dengan tepat ketika ada validasi yang gagal. Jika ditemukan bug apapun, kamu yang memperbaikinya langsung di kode, lalu dokumentasikan hasilnya. Hasil testing-mu akan dipakai Leader sebagai acuan apakah modul Course & Material siap dikoneksikan ke frontend.

Langkah-Langkah

A. Persiapan Awal (Sinkronisasi Laptop)

Karena sudah banyak perubahan di `develop` (Sprint 1-3 sudah di-merge semua), laptop kamu kemungkinan besar sudah tidak sinkron. Untuk menghindari error, conflict, atau file duplikat, **WAJIB** jalankan perintah di bawah ini secara berurutan:

```
# 1. Pindah ke branch develop
git checkout develop

# 2. Ambil informasi terbaru dari GitHub tanpa menggabungkannya dulu
git fetch origin

# 3. HAPUS semua modifikasi lokal dan paksa laptop menyamakan 100% dengan GitHub
git reset --hard origin/develop

# 4. HAPUS semua file/folder asing atau duplikat yang tidak terdaftar di GitHub
git clean -fd

# 5. Sinkronkan ulang semua library agar tidak error
npm install

# 6. Masuk ke folder API dan sinkronkan database
cd apps/api
npx prisma generate
npx prisma migrate dev

# 7. Buat branch baru khusus untuk tugas kamu
git checkout -b fitur/backend-sinta-testing
```

```
# 8. Jalankan server lokal
npm run start:dev
```

Download dan install **Postman** dari postman.com/downloads jika belum punya.

B. Skenario Testing Course

Buka Postman dan uji seluruh endpoint Course satu per satu. Lakukan simulasi lengkap mengikuti alur ini:

1. Register akun Dosen dan akun Mahasiswa lewat `POST /auth/register` . Login keduanya untuk mendapatkan token masing-masing.
2. Buat Mata Kuliah baru lewat `POST /courses` menggunakan token Dosen. Pastikan berhasil (status 201) dan response menampilkan data course yang dibuat.
3. Lihat daftar semua matkul lewat `GET /courses` . Pastikan course yang baru dibuat muncul di daftar dan ada informasi nama dosen pengampu.
4. Lihat detail satu matkul lewat `GET /courses/[ID course]` . Pastikan response memuat seluruh data relasi: materials, quizzes, assignments, dan labs (walau masih kosong, field-nya harus ada berupa array kosong).
5. Enroll Mahasiswa ke matkul lewat `POST /courses/[ID course]/enroll` menggunakan token Mahasiswa. Pastikan berhasil. Lalu coba enroll lagi, pastikan ditolak dengan error 400 karena sudah terdaftar.
6. Panggil `GET /courses/my` menggunakan token Mahasiswa. Pastikan course yang baru di-enroll muncul di daftar kelasku.
7. Edit matkul lewat `PUT /courses/[ID course]` menggunakan token Dosen pemilik. Pastikan berhasil. Lalu coba edit menggunakan token dosen lain — pastikan ditolak dengan error 403 Forbidden.
8. Hapus matkul lewat `DELETE /courses/[ID course]` . Pastikan berhasil dan mekanisme Cascade Delete bekerja, yaitu semua materi, kuis, tugas, lab, dan data enrollment yang terkait ikut terhapus dari database.

C. Skenario Testing Material

Buat lagi course baru untuk testing Material. Lalu uji seluruh endpoint Material:

1. Upload materi tipe TEXT lewat `POST /materials` . Isi: judul, type "TEXT", konten teks, dan courseId. Pastikan berhasil.
2. Upload materi tipe VIDEO lewat `POST /materials` . Isi: judul, type "VIDEO", dan URL video. Pastikan berhasil.
3. Upload materi tipe DOCUMENT lewat `POST /materials` . Isi: judul, type "DOCUMENT", dan URL file. Pastikan berhasil.
4. Lihat detail satu materi lewat `GET /materials/[ID materi]` . Pastikan response menampilkan judul, tipe, dan konten/URL secara utuh.
5. Edit materi lewat `PUT /materials/[ID materi]` menggunakan token Dosen pemilik. Pastikan data berubah. Lalu coba edit menggunakan token dosen lain, pastikan error 403.
6. Hapus materi lewat `DELETE /materials/[ID materi]` menggunakan token Dosen pemilik. Pastikan data terhapus dari database.

D. Bug Fixing

Jika ditemukan error/bug saat testing, buka file service yang relevan di folder `apps/api/src/course/` atau `apps/api/src/material/`. Perbaiki masalahnya langsung. Fokus pada: response JSON yang tidak rapi (field yang seharusnya tidak muncul), pesan error HTTP yang salah kode-nya (misal seharusnya 404 tapi malah 500), dan validasi yang terlewat.

E. Dokumentasi Hasil

Buat file baru `apps/api/test/TESTING_RESULTS.md`. Di dalamnya, catat hasil testing kamu dalam bentuk tabel: nomor, endpoint, skenario, status (PASS/FAIL), dan catatan jika ada bug yang diperbaiki. File ini yang akan di-review oleh Leader.

F. Selesai? Push ke GitHub

```
git add .  
git commit -m "feat: testing api course dan material lengkap serta bug fixing"  
git push origin fitur/backend-sinta-testing
```

Buka GitHub, buat **Pull Request ke branch** `develop` (bukan main). Tugaskan Ariel sebagai reviewer.

TUGAS DIMAS - Lab Submission & Grading

Branch: fitur/backend-dimas-lab-submission

 **TUNGGU TIO SELESAI DULU!** Modul kamu bergantung pada modul Lab CRUD milik Tio. Jangan mulai coding sebelum PR Tio sudah di-merge ke `develop` oleh Leader. Setelah di-merge, jalankan ulang perintah sinkronisasi di halaman pertama panduan ini sebelum membuat branch baru.

Deskripsi

Setelah Dosen membuat sesi praktikum (dikerjakan Tio), giliran kamu membangun sisi mahasiswa dan penilaiannya. Konsepnya sama persis dengan fitur Submission & Grading pada modul Assignment (tugas rutin) yang sudah ada di `apps/api/src/assignment/`, bedanya kali ini untuk praktikum (Lab). Kamu membuat 3 endpoint utama: jalur pengumpulan hasil praktikum oleh mahasiswa, halaman dosen untuk melihat siapa saja yang sudah mengumpulkan, dan fitur penilaian agar dosen bisa memberikan skor dan komentar. API yang kamu buat nanti akan dipakai Frontend untuk merender form submit hasil lab, tabel daftar submission, dan form grading dosen.

Langkah-Langkah

A. Persiapan Awal (Sinkronisasi Laptop)

 **Pastikan PR Tio sudah di-merge ke `develop` sebelum memulai.** Tanya Leader kalau belum yakin.

Karena sudah banyak perubahan di `develop` (Sprint 1-3 + kode Tio sudah di-merge), laptop kamu kemungkinan besar sudah tidak sinkron. Untuk menghindari error, conflict, atau file duplikat, **WAJIB** jalankan perintah di bawah ini secara berurutan:

```
# 1. Pindah ke branch develop
git checkout develop

# 2. Ambil informasi terbaru dari GitHub tanpa menggabungkannya dulu
git fetch origin

# 3. HAPUS semua modifikasi lokal dan paksa laptop menyamakan 100% dengan GitHub
git reset --hard origin/develop

# 4. HAPUS semua file/folder asing atau duplikat yang tidak terdaftar di GitHub
git clean -fd

# 5. Sinkronkan ulang semua library agar tidak error
npm install

# 6. Masuk ke folder API dan sinkronkan database
cd apps/api
npx prisma generate
npx prisma migrate dev
```

```
# 7. Buat branch baru khusus untuk tugas kamu
git checkout -b fitur/backend-dimas-lab-submission
```

B. Database

Model `LabSubmission` sudah dibuat oleh Tio di Sprint ini. Kamu **tidak perlu mengubah database**. Cukup pastikan setelah kamu `git pull` dari develop terbaru, model `LabSubmission` sudah ada di file `schema.prisma` dengan field-field yang dibutuhkan: `id`, `labId`, `studentId`, `fileUrl`, `note`, `status`, `score`, `feedback`. Kalau belum ada, berarti PR Tio belum di-merge — tanya Leader dulu.

C. Mulai Coding

Buat modul baru di folder `apps/api/src/lab-submission/`. Gunakan generator NestJS (jalankan dari folder `apps/api`) untuk membuat module, controller, dan service untuk `lab-submission`. Pastikan `PrismaService` terdaftar di providers modul tersebut. Kamu bisa lihat modul `assignment` di folder `apps/api/src/assignment/` sebagai referensi pola kode — konsepnya sangat mirip.

1. **Buka file service (`lab-submission.service.ts`).** Import `PrismaService` dan masukkan ke constructor. Kamu harus membuat 3 fungsi utama:

- **Fungsi `submit`** - Menerima ID lab (dari parameter URL), `userId` (dari token JWT), `fileUrl` (link file hasil praktikum), dan `note` (catatan tambahan, opsional). Lakukan 3 validasi berturut-turut sebelum menyimpan:
 - **Validasi 1:** Cek lab-nya ada di database. Kalau tidak ada, kasih error `NotFoundException`.
 - **Validasi 2:** Cek mahasiswa terdaftar di kelas lab tersebut. Query ke tabel `Enrollment` menggunakan kombinasi `userId` dan `courseId` milik lab. Kalau tidak terdaftar, tolak dengan `ForbiddenException`. Ini mencegah penyusup yang bukan anggota kelas mengumpulkan hasil.
 - **Validasi 3:** Cek apakah mahasiswa sudah pernah submit sebelumnya untuk lab ini. Query ke tabel `LabSubmission` menggunakan kombinasi `labId` dan `studentId`. Kalau sudah ada, tolak dengan `BadRequestException`. Ini mencegah pengumpulan ganda.Kalau ketiga validasi lolos, simpan data submission baru ke tabel `LabSubmission` dengan status awal `"PENDING"`.
- **Fungsi `getSubmissions`** - Menerima ID lab dan `userId` (dosen). Tujuannya agar dosen bisa melihat kompilasi siapa saja yang sudah mengumpulkan hasil praktikum. Cek dulu bahwa lab-nya ada dan `userId` adalah dosen pemilik course. Lalu query seluruh data submission untuk lab tersebut, termasuk nama lengkap dan email mahasiswanya (gunakan relasi `include` ke tabel `User`). Return array berupa nama mahasiswa, tautan file, stempel waktu pengumpulan, status penilaian (`PENDING/GRADED`), dan skor jika sudah dinilai.
- **Fungsi `grade`** - Menerima ID submission, `userId` (dosen), `score` (angka 0-100), dan `feedback` (teks komentar evaluatif, opsional). Cek submission ada di database. Cek bahwa `userId` adalah dosen pemilik course dari lab tersebut (ini membutuhkan query relasi berantai: `submission > lab > course > instructorId`). Update data submission: simpan nilai `score`, simpan teks `feedback`, dan ubah status dari `"PENDING"` menjadi `"GRADED"`. Setelah disimpan, mahasiswa akan bisa melihat nilainya secara transparan.

2. **Buka file controller (`lab-submission.controller.ts`).** Karena endpoint-mu memiliki awalan URL yang berbeda-beda (`/labs/...` untuk submit dan lihat submission, serta `/lab-submissions/...` untuk grading), atur dekorator `@Controller()` tanpa prefix agar kamu bisa menulis path lengkap di setiap route. Buat 3 route:

- `POST /labs/:id/submit` - Mahasiswa kumpul hasil lab. Pasang guard JWT.

- `GET /labs/:id/submissions` - Dosen lihat daftar submission. Pasang guard JWT.
- `PUT /lab-submissions/:id/grade` - Dosen kasih nilai. Pasang guard JWT.

3. **Pastikan modul terdaftar.** Cek `apps/api/src/app.module.ts`, pastikan modul `LabSubmissionModule` sudah ada di `imports`.

D. Selesai? Push ke GitHub

```
git add .
git commit -m "feat: lab submission dan grading praktikum lengkap"
git push origin fitur/backend-dimas-lab-submission
```

Buka GitHub, buat **Pull Request ke branch `develop`** (bukan `main`). Tugaskan Ariel sebagai reviewer.

Reminder Final

- **Clean Code:** Tulis struktur kode yang rapi (penamaan variabel logis), tangani error (misal: `NotFoundException`, `ForbiddenException`), dan bersihkan `console.log` sebelum commit.
- Selalu gunakan pola eksekusi asinkron (`async` / `await`) saat membungkus panggilan operasi ke layanan database Prisma.
- Jika kamu mengalami Merge Conflict saat melakukan integrasi atau `git pull`, jangan mengambil risiko menyelesaikan konflik secara paksa jika kamu ragu. Segera komunikasikan di grup atau hubungi Leader.
- **Deadline Sprint 4: Rabu, 28 Mei 2026.** Targetkan penyelesaian coding sebelum tanggal tersebut untuk menyisakan waktu bagi proses peninjauan kode (code review).

Dokumen panduan ini disusun sedemikian rupa untuk memastikan sinkronisasi teknis mendalam dan pemahaman logika bisnis antar seluruh punggawa tim backend Ruang Dosen.